

1. วัตถุประสงค์ของใบงาน

เมื่อทำใบงานนี้เสร็จ นักเรียนจะสามารถ

1. อธิบายหลักการทำงานของ Stack แบบ LIFO และ Queue แบบ FIFO ได้
2. อธิบายหน้าที่ของตัวแปร SP, Front และ Rear ได้
3. ปรับปรุงโค้ดเดิมให้แบ่งเป็นฟังก์ชันอย่างเป็นระบบได้
4. รวมส่วนของโปรแกรมที่ทำงานเหมือนกันให้สามารถใช้ร่วมกันได้
5. แสดงสถานะของ Stack และ Queue ให้ผู้ใช้ติดตามการเปลี่ยนแปลงของข้อมูลได้ง่าย
6. ทดสอบกรณีปกติ กรณีข้อมูลเต็ม และกรณีไม่มีข้อมูลได้

2. คำชี้แจง

ให้นักศึกษาศึกษาโค้ดโปรแกรม Stack และ Queue จากเอกสารประกอบการสอนก่อนเริ่มปฏิบัติ โดยให้สังเกตส่วนประกอบต่อไปนี้

1. ตัวแปร Array ที่ใช้เก็บข้อมูล
2. ตัวแปรที่ใช้ระบุตำแหน่งข้อมูล
3. ฟังก์ชันเพิ่มข้อมูล
4. ฟังก์ชันนำข้อมูลออก
5. การตรวจสอบ Overflow
6. การตรวจสอบ Underflow
7. ฟังก์ชันแสดงข้อมูล
8. เมนูรับคำสั่งจากผู้ใช้

จากนั้นให้นำแนวคิดของโปรแกรมทั้งสองมาพัฒนาเป็นโปรแกรมใหม่ โดยโปรแกรม Stack และ Queue ต้องอยู่ในโปรแกรมเดียวกัน และต้องลดการเขียนโค้ดที่ซ้ำกัน

งานที่ต้องส่ง

ให้นักศึกษาส่งไฟล์ดังต่อไปนี้

1. ไฟล์ Source Code ภาษา C นามสกุล .c
2. ไฟล์รายงานผลการทดลองนามสกุล .pdf
3. ภาพผลลัพธ์การทดสอบครบทั้ง 7 ชุด
4. ตารางเปรียบเทียบ Stack และ Queue
5. คำตอบคำถามท้ายใบงาน
6. แผนภาพแสดงโครงสร้างฟังก์ชันของโปรแกรม
7. คำอธิบายฟังก์ชันที่ Stack และ Queue ใช้ร่วมกัน

ตั้งชื่อไฟล์ดังนี้

Lab03_รหัสนักศึกษา_ชื่อ.pdf

Lab03_รหัสนักศึกษา_ชื่อ.c

รูปแบบใบงาน

เอกสารใบงานควรมีหัวข้อต่อไปนี้

1. ชื่อใบงาน
2. รหัสนักศึกษา ชื่อ-นามสกุล
3. สรุปหลักการ Stack และ Queue
4. โครงสร้างโปรแกรม
5. รายชื่อฟังก์ชันและหน้าที่
6. Source Code
7. ผลการทดสอบ
8. คำตอบคำถามท้ายใบงาน
9. ปัญหาที่พบและวิธีแก้ไข
10. สรุปผลการปฏิบัติ

3. โจทย์ปฏิบัติ

งานที่ 3.1 ศึกษาและทดลองโค้ดต้นฉบับ

ให้นักศึกษานำโค้ดโปรแกรม Stack และ Queue จากเอกสารมาทดลองรัน แล้วตอบคำถามต่อไปนี้

1. Stack ใช้ตัวแปรใดระบุตำแหน่งข้อมูลบนสุด
2. Queue ใช้ตัวแปรใดระบุตำแหน่งหัวคิวและท้ายคิว
3. ฟังก์ชันใดใช้เพิ่มข้อมูลเข้าสู่ Stack
4. ฟังก์ชันใดใช้นำข้อมูลออกจาก Stack
5. ฟังก์ชันใดใช้เพิ่มข้อมูลเข้าสู่ Queue
6. ฟังก์ชันใดใช้นำข้อมูลออกจาก Queue
7. Overflow เกิดขึ้นเมื่อใด
8. Underflow เกิดขึ้นเมื่อใด
9. Stack และ Queue มีลำดับการนำข้อมูลออกแตกต่างกันอย่างไร
10. โค้ดส่วนใดของโปรแกรม Stack และ Queue มีลักษณะคล้ายกัน

งานที่ 3.2 รวมโปรแกรม Stack และ Queue

ให้นักศึกษาสร้างโปรแกรมภาษา C จำนวน 1 โปรแกรม ที่สามารถเลือกใช้งานได้ทั้ง Stack และ Queue เมื่อเริ่มโปรแกรม ให้แสดงเมนูหลักดังนี้

```
=====
                        STACK AND QUEUE PROGRAM
=====
1. Stack
2. Queue
0. Exit
-----
Select menu:
เมื่อเลือก 1 ให้เข้าสู่เมนู Stack
===== STACK MENU =====
1. Push
2. Pop
3. Show Stack
4. Clear Stack
0. Back to Main Menu
-----
Select menu:
เมื่อเลือก 2 ให้เข้าสู่เมนู Queue
===== QUEUE MENU =====
1. Insert
2. Delete
3. Show Queue
4. Clear Queue
0. Back to Main Menu
-----
Select menu:
```

งานที่ 3.3 จัดโครงสร้างโปรแกรมให้ใช้ได้ร่วมกัน

ให้นักศึกษาปรับโครงสร้างโปรแกรมโดยแยกการทำงานเป็นฟังก์ชันอย่างชัดเจน โปรแกรมต้องมีฟังก์ชันอย่างน้อยดังต่อไปนี้

ฟังก์ชันของ Stack

```
void push(int value);  
int pop(void);  
void showStack(void);  
void clearStack(void);  
int isStackFull(void);  
int isStackEmpty(void);
```

ฟังก์ชันของ Queue

```
void insertQueue(int value);  
int deleteQueue(void);  
void showQueue(void);  
void clearQueue(void);  
int isQueueFull(void);  
int isQueueEmpty(void);
```

ฟังก์ชันที่ Stack และ Queue ใช้ร่วมกัน

ให้นักศึกษาสร้างฟังก์ชันกลางอย่างน้อย 2 ฟังก์ชัน เช่น

```
void showProgramHeader(const char title[]);  
void showOperationResult(const char operation[], int value, int success);
```

หรืออาจสร้างฟังก์ชันสำหรับ

- แสดงเส้นคั่น
- แสดงข้อความสถานะ
- รับค่าตัวเลขจากผู้ใช้
- แสดงหมายเลขตำแหน่งของ Array
- ล้างข้อมูลใน Array
- แสดงข้อความผิดพลาด

นักศึกษาต้องอธิบายว่า ฟังก์ชันใดเป็นฟังก์ชันที่ Stack และ Queue ใช้ร่วมกัน และช่วยลดโค้ดที่ซ้ำกันอย่างไร

งานที่ 3.4 ปรับปรุงการแสดงผล Stack

ให้แก้ไขฟังก์ชันแสดง Stack เพื่อให้เห็นข้อมูลจากตำแหน่งบนสุดลงมาถึงตำแหน่งล่างสุด

ตัวอย่างการแสดงผล

```
===== STACK STATUS =====  
Capacity : 5  
Data      : 3  
Top       : 2  
Status    : NORMAL
```

```
-----  
TOP -> [30]  index 2  
       [20]  index 1  
       [10]  index 0  
-----
```

ถ้า Stack ว่าง ให้แสดงผลดังนี้

```
===== STACK STATUS =====  
Capacity : 5  
Data      : 0  
Top       : -1  
Status    : EMPTY
```

```
-----  
Stack is empty.  
-----
```

ถ้าผู้ใช้ Push ข้อมูลในขณะที่ Stack เต็ม ให้แสดงข้อความ

Push failed: Stack Overflow

ถ้าผู้ใช้ Pop ข้อมูลในขณะที่ Stack ว่าง ให้แสดงข้อความ

Pop failed: Stack Underflow

งานที่ 3.5 ปรับปรุงการแสดงผล Queue

ให้แก้ไขฟังก์ชันแสดง Queue เพื่อให้เห็นตำแหน่ง Front และ Rear อย่างชัดเจน

ตัวอย่างการแสดงผล

```
===== QUEUE STATUS =====  
Capacity : 5  
Data      : 3  
Front     : 0  
Rear      : 2  
Status    : NORMAL
```

```
-----  
Index :   0   1   2   3   4  
Data  : [10] [20] [30] [ ] [ ]  
          F               R
```

```
-----  
Order : 10 -> 20 -> 30
```

ถ้า Queue ว่าง ให้แสดงผลดังนี้

```
===== QUEUE STATUS =====  
Capacity : 5  
Data      : 0  
Front     : 0  
Rear      : -1  
Status    : EMPTY
```

```
-----  
Queue is empty.
```

```
-----  
ถ้าผู้ใช้ Insert ข้อมูลในขณะที่ Queue เต็ม ให้แสดงข้อความ
```

Insert failed: Queue Overflow

```
ถ้าผู้ใช้ Delete ข้อมูลในขณะที่ Queue ว่าง ให้แสดงข้อความ
```

Delete failed: Queue Underflow

งานที่ 3.6 แสดงผลก่อนและหลังการทำงาน

ทุกครั้งที่ใช้เลือก Push, Pop, Insert หรือ Delete โปรแกรมต้องแสดงข้อมูลต่อไปนี้

1. ชื่อคำสั่งที่กำลังทำงาน
2. ค่าข้อมูลที่เพิ่มหรือนำออก
3. สถานะก่อนทำงาน
4. สถานะหลังทำงาน
5. ผลการทำงานว่าสำเร็จหรือไม่สำเร็จ

ตัวอย่างการ Push

```
Operation : PUSH
Input      : 30
Before     : Top = 1, Data = 2
After      : Top = 2, Data = 3
Result     : SUCCESS
```

ตัวอย่างการ Pop

```
Operation : POP
Output     : 30
Before     : Top = 2, Data = 3
After      : Top = 1, Data = 2
Result     : SUCCESS
```

ตัวอย่างการ Insert Queue

```
Operation : INSERT QUEUE
Input      : 40
Before     : Front = 0, Rear = 2, Data = 3
After      : Front = 0, Rear = 3, Data = 4
Result     : SUCCESS
```


4. เงื่อนไขของโปรแกรม

โปรแกรมที่พัฒนาต้องเป็นไปตามเงื่อนไขต่อไปนี้

1. ใช้ภาษา C
2. ใช้ Array เป็นโครงสร้างเก็บข้อมูล
3. กำหนดความจุของ Stack และ Queue ด้วยค่าคงที่ เช่น `#define MAX_SIZE 5`
4. Stack และ Queue ต้องอยู่ในโปรแกรมเดียวกัน
5. โปรแกรมต้องทำงานด้วยเมนู
6. โปรแกรมต้องไม่หยุดทำงานเมื่อเกิด Overflow หรือ Underflow
7. หลังจากแสดงข้อความผิดพลาดแล้ว ต้องกลับไปแสดงเมนูได้
8. ต้องมีคำสั่งล้างข้อมูล Stack และ Queue
9. ต้องตรวจสอบค่าที่ผู้ใช้ป้อน
10. ห้ามเขียนคำสั่งทั้งหมดไว้ในฟังก์ชัน `main()`
11. ต้องแยกฟังก์ชันตามหน้าที่
12. ต้องมีฟังก์ชันที่ Stack และ Queue ใช้ร่วมกันอย่างน้อย 2 ฟังก์ชัน
13. ต้องใส่คำอธิบายในโค้ดเฉพาะส่วนสำคัญ
14. ต้องตั้งชื่อตัวแปรและฟังก์ชันให้สื่อความหมาย

5. ลำดับการทดสอบโปรแกรม

ให้นักศึกษาทดสอบโปรแกรมตามลำดับต่อไปนี้ และบันทึกภาพผลลัพธ์ทุกชุด

ชุดทดสอบที่ 1: Stack ทำงานตามปกติ

กำหนด Stack ขนาด 5 ช่อง

```
Push 10
Push 20
Push 30
Pop
Push 40
Show Stack
```

ให้ตอบว่า

1. ข้อมูลใดถูก Pop ออก
2. ข้อมูลที่เหลือใน Stack เรียงจาก Top ลงมาเป็นอย่างไร
3. ค่า Top สุดท้ายเป็นเท่าใด

ชุดทดสอบที่ 2: Stack Overflow

```
Clear Stack
Push 11
Push 22
Push 33
Push 44
Push 55
Push 66
```

ให้ตอบว่า

1. คำสั่งใดทำให้เกิด Overflow
2. ข้อมูล 66 ถูกเก็บลง Stack หรือไม่
3. จำนวนข้อมูลสุดท้ายใน Stack เท่ากับเท่าใด

ชุดทดสอบที่ 3: Stack Underflow

```
Clear Stack
Pop
```

ให้ตอบว่า

1. เพราะเหตุใดจึงเกิด Underflow
2. โปรแกรมยังสามารถกลับไปแสดงเมนูได้หรือไม่

ชุดทดสอบที่ 4: Queue ทำงานตามปกติ

```
Insert 10  
Insert 20  
Insert 30  
Delete  
Insert 40  
Show Queue
```

ให้ตอบว่า

1. ข้อมูลใดถูก Delete ออก
2. ข้อมูลที่เหลืออยู่ใน Queue เรียงตามลำดับอย่างไร
3. ค่า Front และ Rear สุดท้ายเป็นเท่าใด

ชุดทดสอบที่ 5: Queue Overflow

```
Clear Queue  
Insert 11  
Insert 22  
Insert 33  
Insert 44  
Insert 55  
Insert 66
```

ให้ตอบว่า

1. คำสั่งใดทำให้เกิด Overflow
2. ข้อมูล 66 ถูกเก็บใน Queue หรือไม่
3. จำนวนข้อมูลสุดท้ายใน Queue เท่ากับเท่าใด

ชุดทดสอบที่ 6: Queue Underflow

```
Clear Queue  
Delete
```

ให้ตอบว่า

1. เพราะเหตุใดจึงเกิด Underflow
2. โปรแกรมจัดการกับค่าที่ไม่มีอยู่ใน Queue อย่างไร

ชุดทดสอบที่ 7: เปรียบเทียบ Stack และ Queue

ให้ใช้ข้อมูลชุดเดียวกันคือ

10, 20, 30, 40

ดำเนินการดังนี้

1. เพิ่มข้อมูลทั้งหมดเข้าสู่ Stack
2. นำข้อมูลออกจาก Stack จนครบ
3. เพิ่มข้อมูลทั้งหมดเข้าสู่ Queue
4. นำข้อมูลออกจาก Queue จนครบ
5. บันทึกลำดับข้อมูลที่ออกจากแต่ละโครงสร้าง

ให้สรุปผลในตาราง

โครงสร้าง	ลำดับข้อมูลเข้า	ลำดับข้อมูลออก	หลักการ
Stack	10, 20, 30, 40		
Queue	10, 20, 30, 40		

6. คำถามท้ายใบงาน

ให้นักศึกษาตอบคำถามต่อไปนี้ด้วยภาษาของตนเอง

1. Stack และ Queue แตกต่างกันอย่างไรร
2. เพราะเหตุใด Stack จึงใช้ตัวชี้ตำแหน่งเพียงตัวเดียว
3. เพราะเหตุใด Queue จึงต้องมีทั้ง Front และ Rear
4. Overflow และ Underflow แตกต่างกันอย่างไรร
5. ฟังก์ชัน `isStackFull()` มีหน้าที่อะไร
6. ฟังก์ชัน `isQueueEmpty()` มีหน้าที่อะไร
7. เพราะเหตุใดจึงไม่ควรเขียนคำสั่งทั้งหมดไว้ใน `main()`
8. ส่วนใดของโปรแกรม Stack และ Queue สามารถใช้ร่วมกันได้
9. การแสดงค่า Index ของ Array ช่วยให้เข้าใจโปรแกรมอย่างไร
10. ถ้าต้องเพิ่มขนาด Stack และ Queue จาก 5 เป็น 10 ต้องแก้ไขส่วนใด
11. การ Push และ Pop มี Time Complexity เท่าใด
12. การ Insert และ Delete ของ Queue มี Time Complexity เท่าใด
13. ยกตัวอย่างการใช้งาน Stack ในชีวิตจริงหรือระบบคอมพิวเตอร์ 2 ตัวอย่าง
14. ยกตัวอย่างการใช้งาน Queue ในชีวิตจริงหรือระบบคอมพิวเตอร์ 2 ตัวอย่าง

7. ข้อเสนอแนะแนวทางการปฏิบัติ

นักศึกษาควรเริ่มจากทำให้โปรแกรม Stack และ Queue ทำงานได้ถูกต้องแยกกันก่อน จากนั้นจึงนำมารวมในโปรแกรมเดียว แล้วจึงปรับส่วนแสดงผลและฟังก์ชันที่ใช้ร่วมกัน ไม่ควรคัดลอกโค้ดเดิมทั้งหมดโดยไม่วิเคราะห์ นักศึกษาต้องสามารถอธิบายได้ว่า

- ข้อมูลถูกเก็บที่ตำแหน่งใด
- ตัวชี้ตำแหน่งเปลี่ยนแปลงอย่างไร
- เพราะเหตุใดจึงเกิด Overflow
- เพราะเหตุใดจึงเกิด Underflow
- Stack และ Queue นำข้อมูลออกต่างกันอย่างไร